

# Module 4- Test driven development and Logging framework

## Exercise 1: Setting Up JUnit

### Scenario:

You need to set up JUnit in your Java project to start writing unit tests.

### Steps:

1. Create a new Java project in your IDE (e.g., IntelliJ IDEA, Eclipse).
2. Add JUnit dependency to your project. If you are using Maven, add the following to your

pom.xml:

```
<dependency>
```

```
<groupId>junit</groupId>
```

```
<artifactId>junit</artifactId>
```

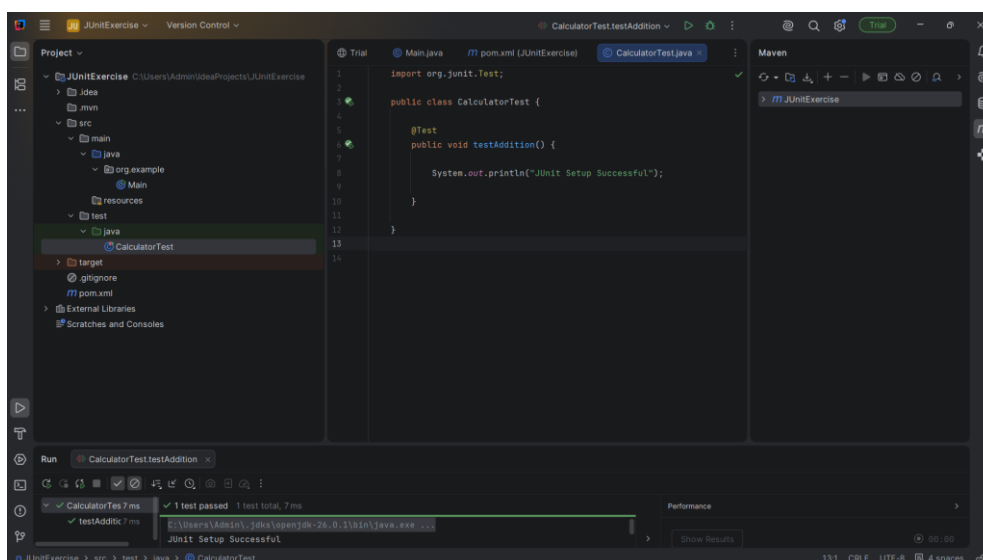
```
<version>4.13.2</version>
```

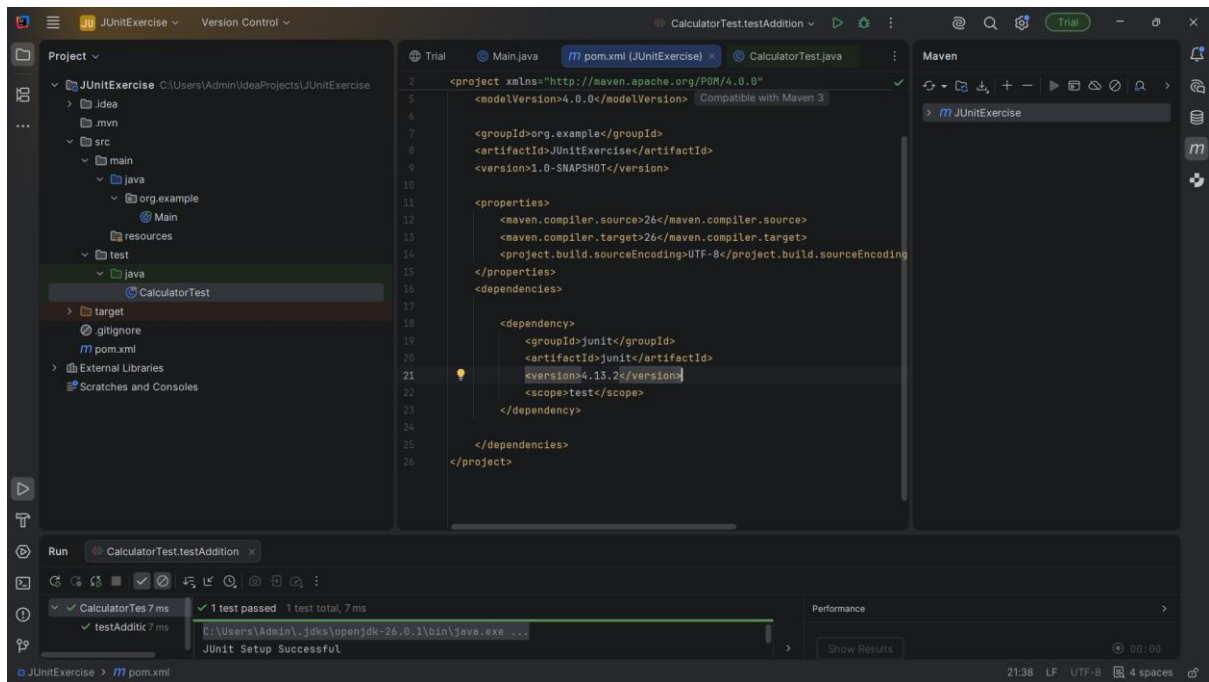
```
<scope>test</scope>
```

```
</dependency>
```

3. Create a new test class in your project.

SOLN::





### Exercise 3: Assertions in JUnit

Scenario:

You need to use different assertions in JUnit to validate your test results.

Steps:

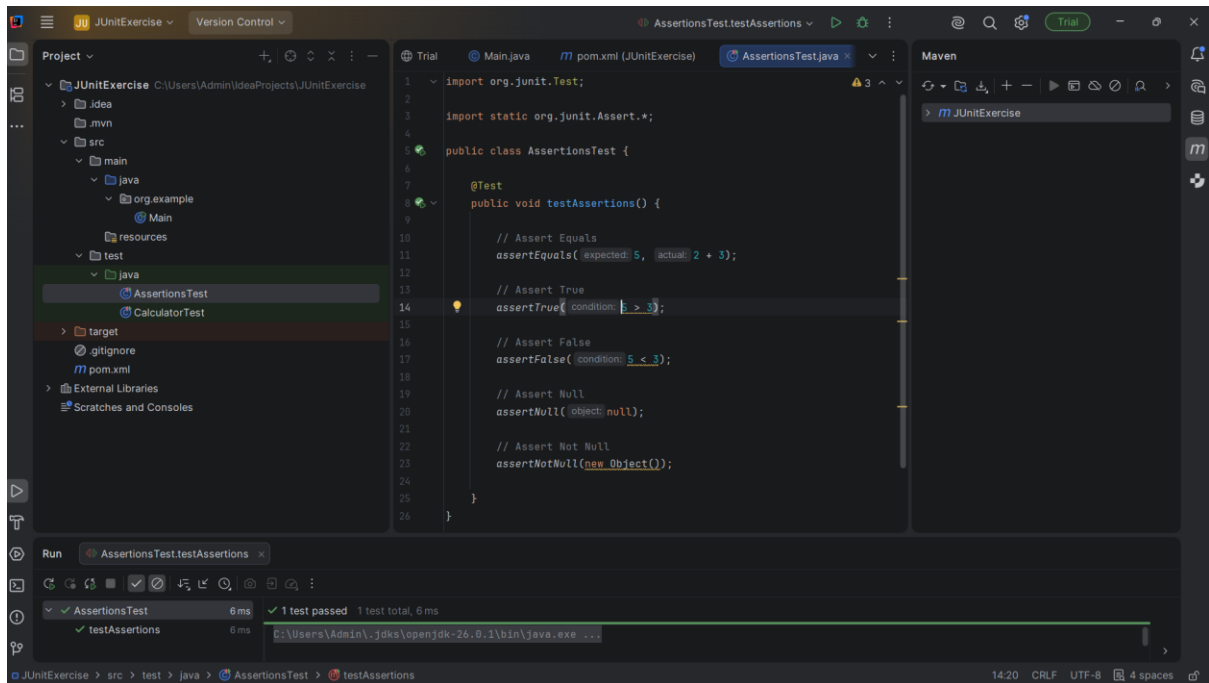
1. Write tests using various JUnit assertions.

Solution Code:

```
public class AssertionsTest {  
    @Test  
    public void testAssertions() {  
        // Assert equals  
        assertEquals(5, 2 + 3);  
        // Assert true  
        assertTrue(5 > 3);  
        // Assert false  
        assertFalse(5 < 3);  
        // Assert null  
        assertNull(null);  
        // Assert not null
```

```
assertNotNull(new Object());  
  
}  
  
}
```

SOLN::



#### Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

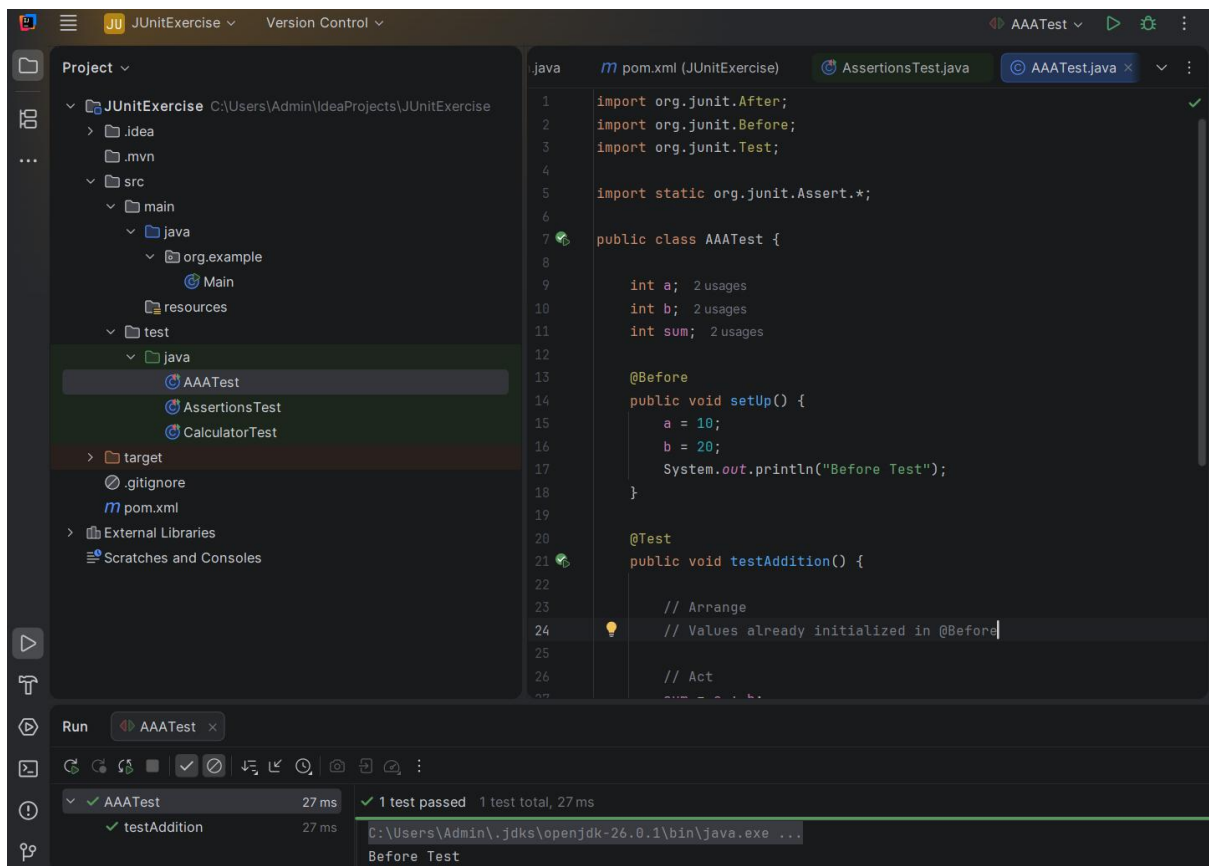
Scenario:

You need to organize your tests using the Arrange-Act-Assert (AAA) pattern and use setup and teardown methods.

Steps:

1. Write tests using the AAA pattern.
2. Use `@Before` and `@After` annotations for setup and teardown methods.

SOLN::



## Exercise 1: Mocking and Stubbing

Scenario:

You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.

Steps:

1. Create a mock object for the external API.
2. Stub the methods to return predefined values.

3. Write a test case that uses the mock object.

Solution Code:

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testExternalApi() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");
        MyService service = new MyService(mockApi);
        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}
```

## Exercise 2: Verifying Interactions

Scenario:

You need to ensure that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Verify the interaction.

Solution Code:

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testVerifyInteraction() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        MyService service = new MyService(mockApi);
```

```
service.fetchData();  
verify(mockApi).getData();  
}  
}
```

JUnitExercise

Version Control

JUnitExercise

Main.java

ExternalApi.java

MyService.java

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

import org.junit.Test;

import static org.junit.Assert.assertEquals;

import static org.mockito.Mockito.\*;

public class MyServiceTest {

@Test

public void testExternalApi() {

ExternalApi mockApi = mock(ExternalApi.class);

when(mockApi.getData()).thenReturn(value: "Mock Data");

MyService service = new MyService(mockApi);

}

}

Run

MyServiceTest.testExternalApi

MyServ

1 sec 652 ms

1 test passed

1 test total, 1 sec 652 ms

testE

1 sec 652 ms

"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...

WARNING: A Java agent has been loaded dynamically (C:\Us

WARNING: If a serviceability tool is in use, please run

WARNING: If a serviceability tool is not in use, please

WARNING: Dynamic loading of agents will be disallowed by

Java HotSpot(TM) 64-Bit Server VM warning: Sharing is on

Process finished with exit code 0

JUnitExercise

>

src

>

test

>

java

>

MyServiceTest

>

testExternalApi

## Exercise 1: Logging Error Messages and Warning Levels

Task: Write a Java application that demonstrates logging error messages and warning levels using SLF4J.

### Step-by-Step Solution:

1. Add SLF4J and Logback dependencies to your `pom.xml` file:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version>
</dependency>
```

2. Create a Java class that uses SLF4J for logging:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {
  private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);

  public static void main(String[] args) {
    logger.error("This is an error message");
    logger.warn("This is a warning message");
  }
}
```



Soln::

The screenshot displays the IntelliJ IDEA IDE interface for a project named "JUnitExercise".

- Project View:** Shows the directory structure: `JUnitExercise` (C:\Users\Admin\IdeaProjects\JUnitExercise) containing `.idea`, `.mvn`, `src`, and `main` directories, with `java` files in `main`.
- Structure View:** Shows the class `LoggingExample` with two methods: `logger: Logger = LoggerFactory.getLogger(...)` and `main(String[]): void`.
- Run View:** Shows the execution of `LoggingExample`. The output is:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program File
18:31:49.790 [main] ERROR org.example.LoggingExample - This is an error mess
18:31:49.796 [main] WARN org.example.LoggingExample - This is a warning mess
Process finished with exit code 0
```

The bottom status bar shows the current file path: `JUnitExercise > src > main > java > org > example > LoggingExample`.